RUSTSCAN MASTERY GUIDE (Advanced + Practical)

SECTION 1: CORE CONCEPT
RustScan is a high-speed TCP port scanner written in Rust. It solves the slowness of traditional scanner

---

SECTION 2: ALL CORE COMMANDS

rustscan -a <target>
→ Scan default top ports
Example:
rustscan -a 10.10.10.10

Expected Output:
[~] Open ports: 22, 80

---

rustscan -a <target> -p <ports>
→ Specify ports
Example:
rustscan -a 10.10.10.10 -p 22,80,443

---

rustscan -a <target> -r <range>
→ Scan range
Example:
rustscan -a 10.10.10.10 -r 1-65535

---

rustscan -a <target> --ulimit <num>
→ Increase performance
Example:
rustscan -a 10.10.10.10 --ulimit 5000

---

rustscan -a <target> --timeout <ms>
→ Set timeout
Example:
rustscan -a 10.10.10.10 --timeout 2000

---

rustscan -a <target> --batch-size <num>
→ Control number of ports scanned at once
Example:
rustscan -a 10.10.10.10 --batch-size 1000

---

rustscan -a <target> -- -sV
→ Pass arguments to Nmap
Example:
rustscan -a 10.10.10.10 -- -sV -sC

Expected Output:
```
PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 8.2
80/tcp open  http    Apache
```

SECTION 3: OUTPUT INTERPRETATION

RustScan output:

→ Fast port discovery

Nmap output:
→ Service detection
→ Versioning
→ Scripts

SECTION 4: EXTREME USAGE (REAL WORLD)

1. FULL RECON PIPELINE:
rustscan -a <target> -r 1-65535 -- -sV -sC -A

2. CTF SPEED RUN:
rustscan -a <target> --ulimit 5000 -- -sV

3. STEALTHIER APPROACH:
Lower batch size:
--batch-size 200

4. SCRIPTING PIPELINE:
ports=$(rustscan -a target | grep Open | cut ...)
nmap -p $ports target

SECTION 5: EDGE CASES

- Firewalled hosts → use slower batch
- VPS scanning → increase ulimit
- Detection evasion → combine with proxychains

SECTION 6: COMPARISON WITH NMAP

RustScan:
✔ Fast discovery
✘ No deep scanning

Nmap:
✔ Deep analysis
✘ Slower

Best practice: Use BOTH

SECTION 7: PRO TIPS

- Always scan ALL ports first
- Never trust default scans
- Automate workflows
- Combine with:
  ffuf, gobuster, nikto

SECTION 8: FINAL WORKFLOW (HTB STYLE)

1. rustscan full scan
2. nmap detailed scan
3. service enumeration
4. exploit development

This guide is optimized for penetration testers, CTF players, and security engineers.